Formal Runtime Enforcement for Safe and Network-Aware Path Planning with Deep Reinforcement Learning*

Ayush Anand[1,2], Aakash[1], Ajay Kattepur[1], Swarup Mohalik[1], and Srinivas Pinisetty[2]

[1] Ericsson Research, India
{aakash.b,ajay.kattepur,swarup.kumar.mohalik}@ericsson.com
[2] Indian Institute of Technology Bhubaneswar, India
{a23cs09003,spinisetty}@iitbbs.ac.in

A Appendix

A.1 Description of Properties

Figure 1 depicts the Discrete Timed Automata (DTA) $\mathcal{A}_{\varphi_1}, \ldots, \mathcal{A}_{\varphi_8}$ corresponding to the eight safety and network-aware properties listed in Table 1. In each automaton, l_0 denotes the initial (accepting/safe) location, while locations such as l_1 in figures 1a, 1b, 1c, 1d and 1e (and l_2 in figures 1f, 1g, and 1h) represent the violating location. The symbol a_d denotes the direction action taken by the agent. The DTAs $\mathcal{A}_{\varphi_6} - \mathcal{A}_{\varphi_8}$ represent *regular discrete timed properties*, where $\{c_1\}, \{c_2\}$ and $\{c_3\}$ are the discrete clock sets of $\mathcal{A}_{\varphi_6}, \mathcal{A}_{\varphi_7}$, and $\mathcal{A}_{\varphi_8}$ respectively.

φ_1 *(Wall/Obstacle Avoidance).* The automaton $\mathcal{A}_{\varphi_1}$ enforces collision avoidance. As long as the chosen direction a_d does not lead into a wall ($\neg wall_d$), the system remains in the safe location l_0. If the action drives the agent into a wall ($wall_d$), the automaton transitions to the violating location l_1.

φ_2 *(No Recurrence / Loop Avoidance).* The automaton $\mathcal{A}_{\varphi_2}$ prevents the agent from revisiting a previously occupied state, thereby avoiding cyclic or oscillatory behavior. The system stays in l_0 when the action does not cause recurrence ($\neg recur_d$); a recurring move ($recur_d$) leads to the violating location l_1.

φ_3 *(Signal-to-Noise Ratio Safety).* The automaton $\mathcal{A}_{\varphi_3}$ forbids moving into a direction with poor connectivity. The system remains safe in l_0 when the action keeps the SNR acceptable ($\neg low\_snr_d$), and transitions to l_1 if the chosen direction results in a low SNR (low_snr_d).

φ_4 *(Throughput Safety).* Analogous to φ_3, the automaton $\mathcal{A}_{\varphi_4}$ prohibits actions that drive the agent into low-throughput regions. The system stays in l_0 while throughput remains adequate ($\neg low\_thp_d$), and moves to the violating state l_1 on a low-throughput action (low_thp_d).

* The first two authors have contributed equally to this work.

φ_5 *(RSRP Safety).* The automaton $\mathcal{A}_{\varphi_5}$ ensures the agent avoids regions of weak reference signal received power. It remains in l_0 for actions where $\neg low\_rsrp_d$ holds and transitions to l_1 when the action leads to low RSRP (low_rsrp_d).

φ_6 *(Timed SNR Recovery).* The automaton $\mathcal{A}_{\varphi_6}$ is a *timed* property that bounds how long the SNR may stay degraded. From l_0, while the SNR is good ($good\_snr_d$), the system self-loops in the safe state. When the SNR first becomes poor ($\neg good\_snr_d$), the clock is reset ($c_1 := 0$) and the automaton moves to the monitoring location l_1. As long as the deadline is respected ($c_1 \leq t_1$), the system may stay in l_1 or return to l_0 once the SNR recovers. However, if the degraded condition persists beyond the deadline ($c_1 > t_1$), the automaton transitions to l_2, from which only a sufficiently strong recovery ($better\_snr_d \wedge \neg good\_snr_d$) returns the system to a non-violating state, while a continued weak signal ($\neg better\_snr_d \wedge \neg good\_snr_d$) drives it to the violating sink l_3.

φ_7 *(Timed Throughput Recovery).* The automaton $\mathcal{A}_{\varphi_7}$ mirrors φ_6 for the throughput metric. While throughput is good ($good\_thp_d$), the system remains in l_0. On degradation ($\neg good\_thp_d$) the clock resets and the system enters l_1, tolerating the low-throughput condition only within the time bound t_2. If $c_2 > t_2$, control passes to l_2, where adequate recovery ($better\_thp_d$) is required to avoid the violating state l_3.

φ_8 *(Timed RSRP Recovery).* The automaton $\mathcal{A}_{\varphi_8}$ is the timed counterpart for RSRP. Good RSRP ($good\_rsrp_d$) keeps the system in l_0; degradation ($\neg good\_rsrp_d$) resets the clock and moves it to l_1, where the low-RSRP condition is permitted only for $c_3 \leq t_3$. Once $c_3 > t_3$, the automaton transitions to l_2, and a violation (l_3) occurs unless a sufficiently improved signal ($better\_rsrp_d$) is observed.

In summary, properties φ_1 and φ_2 enforce *safety and navigation constraints* (obstacle avoidance and loop prevention), while φ_3–φ_5 enforce *instantaneous network-quality constraints* on SNR, throughput, and RSRP, respectively. Properties φ_6–φ_8 are their *timed* extensions, which tolerate transient degradations of the respective network metrics provided they recover within the bounded durations t_1, t_2, and t_3.

A.2 Proof of Remark 3

We recall Definition 4 (the definition of the enforcer) and Remark 3 from the main paper.

Definition 4 (Enforcer of a property φ). *Given a property φ, an enforcer of φ is a function $E_\varphi : \Sigma^* \to \Sigma^*$ that satisfies following constraints:*

1. Soundness- *it means that the output of the enforcer can always be extended to satisfy φ.*
2. Monotonicity- *the enforcer cannot undo what has already been released as output by it.*

Table 1: List of policies

Policy	Description
φ_1	*The autonomous agent must not collide with a wall.*
φ_2	*At any step, the sequence of the last k visited cells must contain at least k/2 distinct cells. This property disallows revisiting a cell more than twice in the last k visited cells, thereby preventing the looping behaviour.*
φ_3	*SNR must be above a minimum threshold* τ_{lo}^{snr}
φ_4	*Throughput must be above a minimum threshold* τ_{lo}^{thp}
φ_5	*RSRP must be above a minimum threshold* $\tau_{lo}^{\mathsf{rsrp}}$
φ_6	*If an SNR is below threshold* τ_{hi}^{snr} *for more than* t_1 *consecutive steps, then the SNR must monotonically increase until the threshold is crossed.*
φ_7	*If an throughput is below threshold* τ_{hi}^{thp} *for more than* t_2 *consecutive steps, then the throughput must monotonically increase until the threshold is crossed.*
φ_8	*If an RSRP is below threshold* $\tau_{hi}^{\mathsf{rsrp}}$ *for more than* t_3 *consecutive steps, then the RSRP must monotonically increase until the threshold is crossed.*

3. Instantaneity- *the enforcer cannot delay, suppress, or insert an event; it can only replace an event.*
4. Transperancy- *the enforcer cannot do unnecessary edits; if the original input of the enforcer can be extended to satisfy φ, then the enforcer must release it unchanged.*
5. Causality*: at each time step, the enforcer first processes the input, then it calls the program to which the input needs to be fed, and finally it processes the output by the program and releases it; this order of execution must be preserved.*

Functionally, given a word $\sigma \cdot (x, y) \in \Sigma^*$, and a property φ expressed as DTA $\mathcal{A}_\varphi$, an enforcer of φ is defined as:

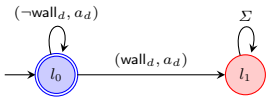
$$E_\varphi(\sigma \cdot (x, y)) = E_\varphi(\sigma) \cdot (x, select(y, edit_\varphi(q, x))) \tag{1}$$

where q is one of the states of $[\![\mathcal{A}_\varphi]\!]$ such that $q_0 \xrightarrow{\sigma} q$, and q_0 is the initial state.

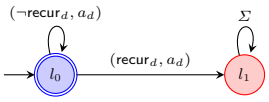
Remark 3. Consider a property φ that is enforceable. Let $\sigma = (I_{val_1}, \mathsf{action}_1) \cdot (I_{val_2}, \mathsf{action}_2) \cdots (I_{val_k}, \mathsf{action}_k)$ be the input of Algorithm 1 where I_{val_t} (resp. action_t) is the input (resp. output) event received by the algorithm in step t in line 12 (resp. 14) for $t = 1, 2, ..., k$. Let $E^*_\varphi(\sigma) = (I_{val_1}, \mathsf{action}'_1) \cdot (I_{val_2}, \mathsf{action}'_2) \cdots (I_{val_k}, \mathsf{action}'_k)$, where $(I_{val_t}, \mathsf{action}'_t)$ is the output released by Algorithm 1 (in line 17), for $t = 1, 2, ..., k$. The function $E^*_\varphi()$ is an enforcer of φ, *i.e.*, it satisfies all the constraints in Definition 4. The proof that $E^*_\varphi()$ is an enforcer of φ as per Definition 4, is given below.

Proof. Let φ be an enforceable property specified as a DTA $A_\varphi = \{L, l_0, \Sigma, C, \Delta, F\}$ with semantics $[\![A_\varphi]\!] = \{Q, q_0, \Sigma, \delta, Q_F\}$, where $\Sigma = \Sigma_I \times \Sigma_O$. Let $Q_v \subseteq Q$ denote the set of states from which no accepting state in Q_F is reachable. Since φ is enforceable, the enforceability condition holds:

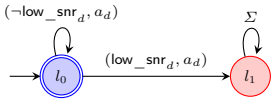
$$\forall q \in Q \setminus Q_v,\ \exists \sigma \in \Sigma^* :\ q \xrightarrow{\sigma} q' \wedge q' \in Q_F. \tag{EnfCo}$$



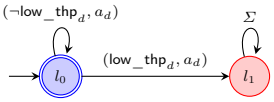
(a) φ_1 expressed as $\mathcal{A}_{\varphi_1}$



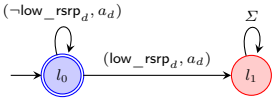
(b) φ_2 expressed as $\mathcal{A}_{\varphi_2}$



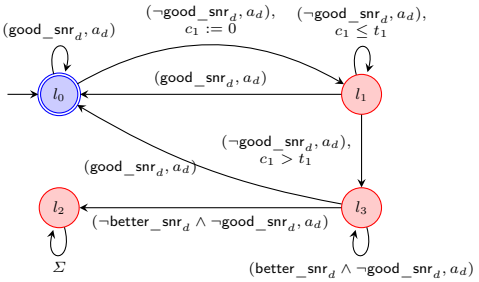
(c) φ_3 expressed as $\mathcal{A}_{\varphi_3}$



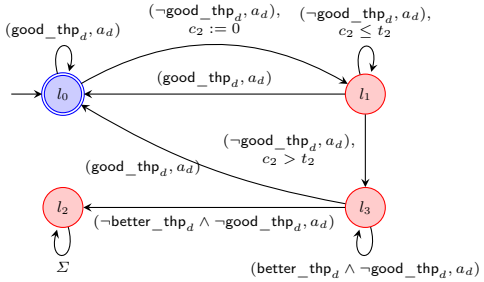
(d) φ_4 expressed as $\mathcal{A}_{\varphi_4}$



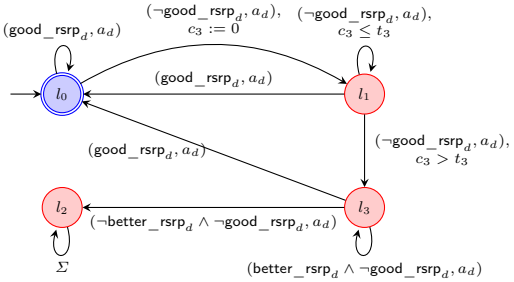
(e) φ_5 expressed as $\mathcal{A}_{\varphi_5}$



(f) φ_6 expressed as $\mathcal{A}_{\varphi_6}$



(g) φ_7 expressed as $\mathcal{A}_{\varphi_7}$



(h) φ_8 expressed as $\mathcal{A}_{\varphi_8}$

Fig. 1: Properties of autonomous vehicle system.

Recall the output-only enforcer (Equation 1):

$$E_\varphi(\sigma \cdot (x, y)) = E_\varphi(\sigma) \cdot (x, \text{select}(y, \text{edit}_\varphi(q, x))),$$

where q is the state reached such that $q_0 \xrightarrow{\sigma} q$, and

$$\text{edit}_\varphi(q, x) = \{\, z \in \Sigma_O \mid \exists \sigma' \in \Sigma^*,\ q \xrightarrow{(x,z)\cdot\sigma'} q'' \wedge q'' \in F \,\}.$$

Let $\sigma = (Ival_1, action_1) \cdots (Ival_k, action_k) \in \Sigma^*$ be the input word consumed by Algorithm 1, and let $E^*_\varphi(\sigma) = (Ival_1, action'_1) \cdots (Ival_k, action'_k)$,

Algorithm 1 Online algorithm for the enforcement of φ

1: Input $\mathsf{position}_{start}, \mathsf{position}_{goal}, MAX\_STEPS, \mathcal{A}_\varphi$
2: $\mathsf{position}_{curr} \leftarrow \mathsf{position}_{start}$
3: $q \leftarrow q_0$ ▷ initialize current state q of the DTA $\mathcal{A}_\varphi$ with its initial state q_0
4: $d \leftarrow \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$
5: **while** $\neg\mathsf{done} \wedge t \leq MAX\_STEPS$ **do**
6: $S_t \leftarrow \mathsf{last\_k\_sequence}()$ ▷ S_t contains the sequence of grid coordinates of last k visited cells
7: $snr_c, pos_d, thp_c, rsrp_c \leftarrow \mathsf{read\_current\_cell}()$
8: **for** each d in $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ **do**
9: $w_d, snr_d, thp_d, rsrp_d \leftarrow \mathsf{read\_neighbour\_cell_d}()$
10: $\mathsf{wall}_d = \Lambda_{wall}(w_d)$, $\mathsf{recur}_d = \Lambda_{\mathsf{recur}}(pos_d, S_t)$, $\mathsf{low\_snr}_d = \Lambda_{\mathsf{safe\_snr}}(snr_d)$, $(\mathsf{good\_snr}_d, \mathsf{better\_snr}_d) = \Lambda_{\mathsf{good\_snr}}(snr_d, snr_c)$, $\mathsf{low\_thp}_d = \Lambda_{\mathsf{safe\_thp}}(thp_d)$, $(\mathsf{good\_thp}_d, \mathsf{better\_thp}_d) = \Lambda_{\mathsf{good\_thp}}(thp_d, thp_c)$, $\mathsf{low\_rsrp}_d = \Lambda_{\mathsf{safe\_rsrp}}(rsrp_d)$, $(\mathsf{good\_rsrp}_d, \mathsf{better\_rsrp}_d) = \Lambda_{\mathsf{good\_rsrp}}(rsrp_d, rsrp_c)$
11: **end for**
12: $I_{val} \leftarrow$ boolean input values ▷ I_{val} is the current valuation of all variables in Σ_I
13: $O'_{actions} \leftarrow edit_\varphi(q, I_{val})$
14: $\mathsf{action}_{cur}, \mathsf{done} \leftarrow RL\_agent(\mathsf{position}_{curr}, \mathsf{position}_{goal})$
15: $\mathsf{action}_{enf} \leftarrow select(\mathsf{action}_{curr}, O'_{actions})$
16: $\mathsf{position}_{curr} \leftarrow execute(\mathsf{position}_{curr}, \mathsf{action}_{enf})$
17: $release(I_{val}, \mathsf{action}_{enf})$
18: $q \leftarrow \delta(q, (I_{val}, \mathsf{action}_{enf}))$
19: $t \leftarrow t + 1$
20: **end while**

where $(Ival_t, action'_t)$ is the event released by Algorithm 1 (line 17) at step $t = 1, \ldots, k$.

We prove that E^*_φ satisfies the five constraints of Definition 4 (Soundness, Monotonicity, Instantaneity, Transparency, Causality) by induction on $|\sigma|$ (equivalently, the number of iterations t of the `while` loop in Algorithm 1).

Induction basis. For $\sigma = \epsilon$, Algorithm 1 releases no event, hence $E^*_\varphi(\epsilon) = \epsilon$, and all constraints hold trivially.

Induction hypothesis. Assume that for every word $\sigma \in \Sigma^*$ of length k, the output $E^*_\varphi(\sigma)$ satisfies all five constraints. Let $q \in Q \setminus Q_v$ be the state reached after consuming $E^*_\varphi(\sigma)$, i.e., $q_0 \xrightarrow{E^*_\varphi(\sigma)} q$. The variable q in Algorithm 1 (lines 3, 18) never belongs to Q_v: it starts at q_0 and is advanced (line 18) only over released reactions, which by construction of edit_φ and (EnfCo) keep an accepting state reachable.

Induction step. Consider a new reaction $(x, y) = (Ival_{k+1}, action_{k+1}) \in \Sigma$, where $x \in \Sigma_I$ is read by Algorithm 1 (lines 6–12) and $y \in \Sigma_O$ is the action proposed by the DDQN agent (line 14). In iteration $k + 1$, Algorithm 1 computes $O'_{actions} = \mathrm{edit}_\varphi(q, x)$ (line 13) and releases $action'_{k+1} = \mathrm{select}(y, O'_{actions})$ (lines 15, 17).

Non-emptiness of $\mathrm{edit}_\varphi(q, x)$. Since $q \in Q \setminus Q_v$, by (EnfCo) there exists $\sigma_0 \in \Sigma^+$ with $q \xrightarrow{\sigma_0} q'$ and $q' \in Q_F$. As A_φ is deterministic and complete, and (by Remark 1 and Remark 2) the observed inputs always admit a satisfying continuation, the output component z of the first reaction (x, z) of such a continuation satisfies $z \in \mathrm{edit}_\varphi(q, x)$. Hence $\mathrm{edit}_\varphi(q, x) \neq \emptyset$ and $\mathrm{select}(y, \mathrm{edit}_\varphi(q, x))$ is well defined.

Case 1: $y \in \text{edit}_\varphi(q, x)$**.** Then $\text{select}(y, \text{edit}_\varphi(q, x)) = y$, so $action'_{k+1} = y$ and $E^*_\varphi(\sigma \cdot (x, y)) = E^*_\varphi(\sigma) \cdot (x, y)$.

- *Soundness.* As $y \in \text{edit}_\varphi(q, x)$, there exists $\sigma' \in \Sigma^*$ with $q \xrightarrow{(x,y)\cdot\sigma'} q''$, $q'' \in F$. Since $q_0 \xrightarrow{E^*_\varphi(\sigma)} q$, $E^*_\varphi(\sigma) \cdot (x, y) \cdot \sigma'$ is accepted by A_φ, so the output can be extended to satisfy φ.
- *Monotonicity.* $E^*_\varphi(\sigma) \preceq E^*_\varphi(\sigma) \cdot (x, y) = E^*_\varphi(\sigma \cdot (x, y))$.
- *Instantaneity.* By hypothesis $|E^*_\varphi(\sigma)| = k$; exactly one event is released, so $|E^*_\varphi(\sigma \cdot (x, y))| = k + 1 = |\sigma \cdot (x, y)|$.
- *Transparency.* As (x, y) keeps the system on a satisfying path, the enforcer forwards it unedited.
- *Causality.* Algorithm 1 reads x (lines 6–12), then queries the agent for y (line 14), then processes and releases the output (lines 15–17); inputs are observed unaltered (input edit is the identity).

Case 2: $y \notin \text{edit}_\varphi(q, x)$**.** The proposed action would lead to a state in Q_v, so the enforcer corrects it. By definition of select,

$$action'_{k+1} = \text{select}(y, \text{edit}_\varphi(q, x)) = \text{rand}(\text{edit}_\varphi(q, x)) \in \text{edit}_\varphi(q, x),$$

which is well defined since $\text{edit}_\varphi(q, x) \neq \emptyset$. (In our implementation select picks the admissible action with maximum Q-value, which is still an element of $\text{edit}_\varphi(q, x)$; the argument is unaffected.) Thus $E^*_\varphi(\sigma \cdot (x, y)) = E^*_\varphi(\sigma) \cdot (x, action'_{k+1})$.

- *Soundness.* Since $action'_{k+1} \in \text{edit}_\varphi(q, x)$, there exists $\sigma' \in \Sigma^*$ with $q \xrightarrow{(x,action'_{k+1})\cdot\sigma'} q''$, $q'' \in F$. As $q_0 \xrightarrow{E^*_\varphi(\sigma)} q$, $E^*_\varphi(\sigma) \cdot (x, action'_{k+1}) \cdot \sigma'$ is accepted by A_φ, so the released output can be extended to satisfy φ.
- *Monotonicity.* $E^*_\varphi(\sigma) \preceq E^*_\varphi(\sigma) \cdot (x, action'_{k+1}) = E^*_\varphi(\sigma \cdot (x, y))$.
- *Instantaneity.* By hypothesis $|E^*_\varphi(\sigma)| = k$; exactly one event is released, so $|E^*_\varphi(\sigma \cdot (x, y))| = k + 1 = |\sigma \cdot (x, y)|$.
- *Transparency.* The original reaction (x, y) cannot be extended to satisfy φ (as $y \notin \text{edit}_\varphi(q, x)$ leads only to Q_v); hence editing is necessary, and Transparency is not violated. Note the input x is preserved; only the output is modified, consistent with output-only enforcement.
- *Causality.* As in Case 1, Algorithm 1 reads the input (lines 6–12), invokes the agent (line 14), and only then processes and releases the (edited) output (lines 15–17). The input is never altered.

In both cases, $E^*_\varphi(\sigma \cdot (x, y))$ satisfies the Soundness, Monotonicity, Instantaneity, Transparency, and Causality constraints, and the state $q'' \in Q \setminus Q_v$ reached upon the released reaction is used to update q in line 18, preserving the induction invariant. Hence the claim holds for $\sigma \cdot (x, y)$.

By induction, E^*_φ satisfies all constraints of Definition 4 for every $\sigma \in \Sigma^*$; therefore the function computed by Algorithm 1 is an enforcer of φ.

A.3 Reward Design for DDQN Agent

The reward function for our DDQN agent is designed to encourage its progress toward the goal while discouraging inefficient or undesirable behavior. The reward received after executing an action at time step t is defined as:

$$r_t = -0.02 + (d_t - d_{t+1}) + r_{\text{goal}} + r_{\text{no\_move}},$$

where d_t and d_{t+1} denote the Euclidean distance between the robot and the goal before and after executing the action, respectively. The goal-reaching reward is given by:

$$r_{\text{goal}} = \begin{cases} 50, & \text{if the robot reaches the goal,} \\ 0, & \text{otherwise,} \end{cases}$$

and the no-movement penalty is defined as:

$$r_{\text{no\_move}} = \begin{cases} -0.3, & \text{if the action results in no movement,} \\ 0, & \text{otherwise.} \end{cases}$$

The constant term -0.02 acts as a step penalty that encourages the shorter paths, $(d_t - d_{t+1})$ encourages movement toward the goal, r_{goal} provides a substantial terminal reward upon reaching the goal, and $r_{\text{no\_move}}$ penalizes actions that do not change the robot's position (e.g., actions that result in collisions with walls or workspace boundaries).

A.4 Curriculum Learning

To facilitate stable learning and improve convergence in the considered large workspace, we employ *Curriculum Learning* [1]. Rather than exposing the agent to the entire workspace from the beginning of training, the complexity of navigation tasks is increased progressively through a sequence of curriculum stages.

Let $\mathcal{F}$ denote the set of free cells in the workspace and let $c = (x_c, y_c)$ denote the center of the workspace. The curriculum consists of a sequence of stages characterized by radii $R = R_1, R_2, \ldots, R_K$, where $R_k < R_{k+1}$. At curriculum stage k, the robot's start and goal locations are sampled from the subset

$$\mathcal{F}_k = \{p = (x, y) \in \mathcal{F} \mid \|p - c\|_2 \leq R_k\},$$

where R_k denotes the curriculum radius associated with stage k. For stages $k > 1$, the selection of start and goal locations is additionally constrained so that they are not both selected from $\mathcal{F}_{k-1}$, i.e.,

$$(x_r, y_r)_k \notin \mathcal{F}_{k-1} \ \vee \ (x_g, y_g)_k \notin \mathcal{F}_{k-1}.$$

The additional sampling constraint ensures that at least one of the sampled locations lies outside the region associated with the (already mastered) preceding curriculum stage. As a result, each curriculum stage introduces previously unseen

navigation scenarios and encourages the agent to explore newly introduced areas of the workspace.

The curriculum stages, $1, 2, \ldots, K$, determined by the curriculum radii, are chosen such that the number of newly introduced cells, i.e., $|\mathcal{F}_k \setminus \mathcal{F}_{k-1}|$, remains approximately constant across successive stages. This results in a gradual and balanced increase in task difficulty as the curriculum progresses. Training begins with a small radius, thereby restricting episodes to relatively short navigation tasks. As training progresses, the radius is gradually increased, exposing the agent to a larger portion of the workspace and consequently more challenging start-goal configurations. The curriculum continues until the sampling region encompasses the entire workspace.

Progression through the curriculum is performance-driven rather than episode-driven. Let

$$s_i = \begin{cases} 1, & \text{if the robot successfully reaches the goal in episode } i, \\ 0, & \text{otherwise.} \end{cases}$$

The success rate SR over a sliding window of the most recent M episodes is computed as: $SR = \frac{1}{M} \sum_{i=1}^{M} s_i$. The agent advances from stage k to stage $k+1$ only when SR $\geq \tau$, where $\tau = 0.95$ is the success threshold used in our experiments. Thus, the curriculum adapts automatically to the learning progress of the agent: easier stages are completed quickly, whereas more difficult stages receive additional training until a satisfactory level of competence is achieved.

A.5 Prioritized Experience Replay

To improve sample efficiency during training, we employ *Prioritized Experience Replay* (PER) [2] instead of uniform experience replay. Unlike conventional replay buffers that sample transitions (s, a, s', r) uniformly, PER prioritizes transitions with large temporal-difference (TD) errors, allowing the agent to focus on experiences from which it can learn the most. Let δ_i denote the TD error associated with transition i. The priority assigned to each transition is computed as: $p_i = (|\delta_i| + \eta)^\alpha$, where η is a small positive constant that prevents transitions from receiving zero priority, and α controls the degree of prioritization. When $\alpha = 0$, the method reduces to uniform sampling. A larger TD error for a transition indicates a greater discrepancy between the predicted and target Q-values, suggesting that the transition is more informative for learning.

Transitions are sampled according to the probability distribution: $P(i) = \frac{p_i}{\sum_k p_k}$, such that transitions with larger priorities are sampled more frequently than transitions with smaller priorities. Newly observed transitions are inserted (in to the replay memory) with the maximum priority currently present in the replay memory, ensuring that each transition is sampled at least once before its priority is updated. Since prioritized sampling introduces bias, importance-sampling (IS) weights are used during optimization to compensate for the resulting non-uniform sampling distribution: $w_i = (N \cdot P(i))^{-\beta}$, where N denotes the number of transitions stored in the replay memory and β controls the

amount of bias correction. The IS weights are normalized by the maximum weight in the sampled minibatch before being applied to the loss function. After each training step, the priorities of sampled transitions are updated using their latest TD errors. Following the implementation proposed in [2], the replay memory is implemented using a 'sum-tree' data structure, enabling efficient sampling and priority updates in $\mathcal{O}(\log N)$ time.

A.6 Hyperparameters

To facilitate reproducibility, Table 2 reports the complete set of hyperparameters used for training the DDQN agent.

Table 2: Hyperparameters used for training the Deep RL agent.

Hyperparameter	**Value**
Learning rate	0.000005
Discount factor	0.995
Gradient clipping norm	20
Network architecture	4–1024–$(2048)^{\times 8}$–1024–4
Activation function	ReLU
Loss function	Smooth L1 (Huber) loss
Optimizer	Adam
Batch size	256
Number of warmup episodes	15
Curriculum radii	[8, 11, 13, 15, 17, 20, Full Workspace]
Maximum episode length per curriculum stage	[100, 150, 500, 600, 650, 700, 750]
Target network update frequency (steps)	[500, 750, 1000, 1250, 1500, 2000, 2500]
Initial exploration rate for each curriculum stage	1.0
Minimum exploration rate for each curriculum stage	0.1
Adaptive exploration decay coefficients	(0.001, 0.004)
Success rate threshold for curriculum progression	0.95
Success-rate sliding window length (episodes)	300
Degree of prioritization (α)	0.6
Importance sampling exponent (β)	0.4
PER priority offset (η)	10^{-6}
Replay buffer size	800000

References

1. Bengio, Y., Louradour, J., Collobert, R., Weston, J.: Curriculum learning. In: ICML (2009)
2. Schaul, T., Quan, J., Antonoglou, I., Silver, D.: Prioritized experience replay. In: International Conference on Learning Representations (ICLR) (2016), `https://arxiv.org/abs/1511.05952`